Executable proof of concept: BPMN calling a REST API

Author: Oleksiy Milner · LinkedIn · GitHub repository

The case-study process is not only drawn, it also runs. In this proof of concept, two steps of the process execute as real REST calls against a mock service built from the OpenAPI spec, and the data changes in a database. All names and data are illustrative; no real company data is used.

What is here

Item	Purpose
`Fix_Internet_problems_with_REST_API_calls.bpmn`	Executable Camunda 8 model with two service tasks (REST Outbound Connector)
`mock-api/`	FastAPI + SQLite service implementing `POST`, `GET` and `PATCH /activities` from the OpenAPI spec
`screenshots/`	Evidence of a successful run

How it works

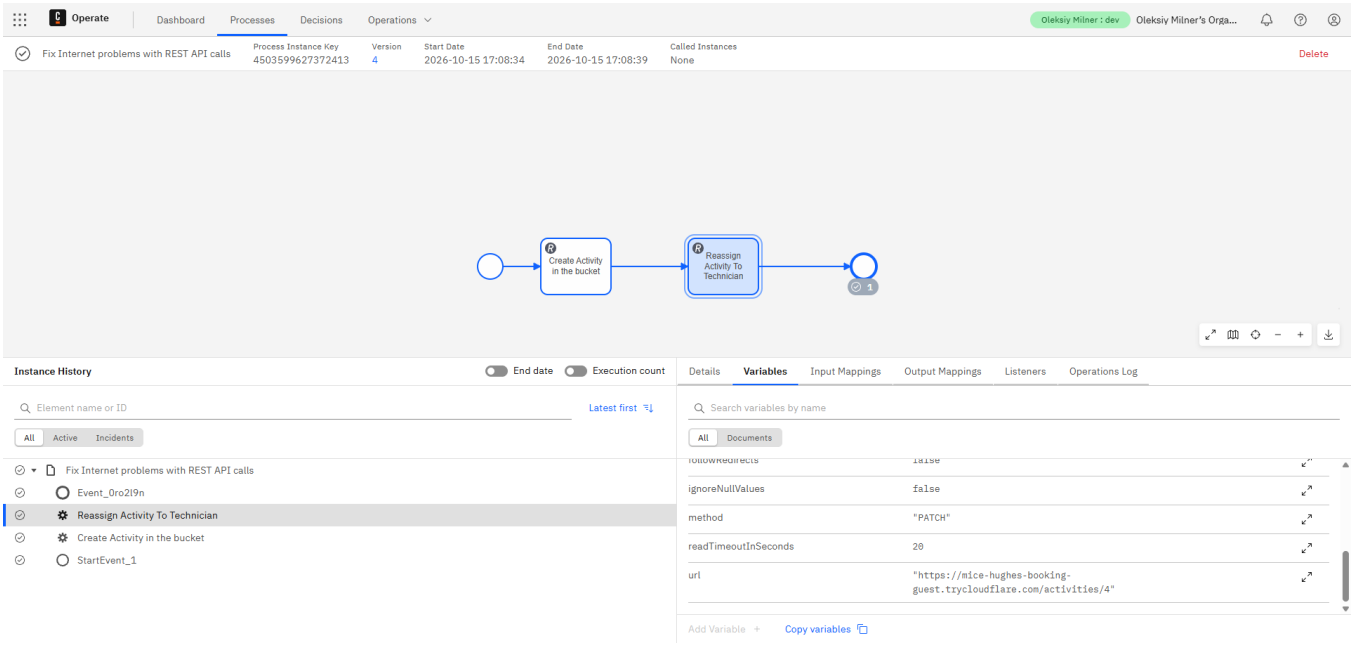
The executable model is a simplified version of the full process: start, two service tasks, end.



Task	REST call	Effect in the database
Create Activity in the Bucket	`POST /activities` with `activityType`, `date`, `resourceId`	A new activity is inserted with status pending in the bucket. The activityId from the response is stored as a process variable.
Reassign Activity To Technician	`PATCH /activities/{activityId}` with `resourceId`	The activity is reassigned to a qualified technician. The service returns 422 if the technician lacks the required work skill.

Evidence

Process instance completed in Camunda Operate. Both service tasks finished, and the URL variable shows the PATCH call to the new activity.



Database after the run. The activity created by the process (row 4) is now assigned to technician 31.



Run it yourself

1. **Start the mock service.** Follow the README in `mock-api/` (Python 3.12 or 3.13). Set the token the process will use before starting:

   ```
   $env:DEMO_TOKEN="your-token"
   ```

2. **Make the service reachable.** Camunda SaaS cannot call localhost, so expose the service with a temporary tunnel, for example:

   ```
   cloudflared tunnel --url http://localhost:8000
   ```

   Alternatively, run Camunda 8 locally and use localhost directly.

3. **Create a connector secret** named DEMO_TOKEN in your Camunda cluster settings, with the same value as the service token. The model reads it as `{{secrets.DEMO_TOKEN}}`, so no token is stored in the file.
4. **Set the URL.** Open the .bpmn file in Camunda Modeler and replace `https://your-service.example.com` in both tasks with your public URL.
5. **Deploy and run** the process, then check the instance in Operate and the rows in the database:

   ```
   python -c "import sqlite3; [print(r) for r in sqlite3.connect('outage.db').execute('select * from activity')]"
   ```

Limitations and next steps

- The bearer-token check is a static token, not real JWT validation. The tunnel is temporary and the database is a single SQLite file; this is a demo setup, not a deployment.
- The executable model covers only the happy path. SLA check, escalation, follow-up visit and customer confirmation stay in the full diagram in the main case-study repository.
- Planned: a run that triggers the `422` skill-mismatch response and handles it with a BPMN error; a third task that sets the status to `completed`; decision tables for the SLA check and technician matching.

Tools

Camunda 8 SaaS (Web Modeler, Operate, REST Outbound Connector), FastAPI, SQLite, Cloudflare quick tunnel.